

RAPPORT DE PROJET SQL

Marketplace LeMauvaisCoin

Conception, Implémentation & Optimisation d'une Base de Données Relationnelle

Alexandre Mulard · Maxime Richard

Projet Final — Cours SQL

SOMMAIRE

1. Modélisation et Schéma Relationnel	3
1.1 Choix du contexte	3
1.2 Entités et Relations	3
1.3 Normalisation (3NF)	4
2. Implémentation Technique	5
2.1 Schéma SQL et Contraintes	5
2.2 Jeu de données	5
3. Requetes Metier Imposees	6
3.1 Jointures et Identification	6
3.2 Agregats et Reporting	6
3.3 Transaction Metier	7
4. Performance et Optimisation	8
4.1 Index CA — Chiffre d'Affaires par vendeur	8
4.2 Index filtre prix + produit (barre de filtre Leboncoin)	8
4.3 Index recherche textuelle (barre de recherche)	9
4.4 Index de performance generale	9
5. Analyse Métier (Interprétation)	9
5.1 Analyse des produits inactifs	10
5.2 Rentabilite des vendeurs	10
Conclusion	10

1. Modélisation et Schéma Relationnel

1.1 Choix du contexte

Nous avons choisi de développer une Marketplace inspirée de Leboncoin, permettant à des utilisateurs de poster des annonces, de catégoriser des produits et d'effectuer des transactions. Le schéma comporte 9 tables : `user`, `type`, `user_type`, `category`, `product`, `sale_ad`, `picture`, `command` et `command_product`. L'extension PostgreSQL `pg_trgm` est activée pour la recherche textuelle avancée sur les titres d'annonces.

1.2 Entités et Relations

Le schéma repose sur les entités clés suivantes :

Entité	Description
<code>user</code>	Profils avec <code>firstname</code> , <code>lastname</code> , <code>email</code> (UNIQUE), <code>hashed_password</code> , <code>birthday</code> . Multi-role gère par la table de liaison <code>user_type</code> (N-N avec <code>type</code>).
<code>sale_ad</code>	Élément central : <code>title</code> , <code>price</code> (DECIMAL 10,2), <code>description</code> , <code>quantity</code> , lie à <code>user</code> et <code>product</code> via clés étrangères CASCADE.
<code>product & category</code>	Organisation hiérarchique 1-N : chaque produit appartient à une catégorie. 10 catégories, dont 3 produits volontairement sans annonces (orphelins).
<code>command</code>	Commande horodatée (TIMESTAMP DEFAULT CURRENT_TIMESTAMP) liée à un acheteur. Produits achetés via <code>command_product</code> (N-N).
<code>picture</code>	Photos d'annonces (plusieurs par annonce) — gestion du carrousel d'images inspire de l'UI Leboncoin.

Schema relationnel de la base de données :



Legende du schema

-  **Cle (key)** — Cle primaire (PRIMARY KEY) : identifiant unique de chaque ligne de la table.
- # Hash / Identity** — Colonne auto-incrementée (SERIAL / IDENTITY) : la valeur est générée automatiquement par la base.
- @ Empreinte (unique)** — Contrainte UNIQUE : deux lignes ne peuvent pas avoir la même valeur (ex : email).
- ◆ Losange plein** — Colonne NOT NULL : la valeur est obligatoire, le champ ne peut pas être vide.
- ◇ Losange vide** — Colonne nullable : la valeur est optionnelle (ex : description, birthday).

1.3 Normalisation (3NF)

Le schéma respecte la 3eme Forme Normale (3NF) :

- Toutes les tables ont une clé primaire SERIAL PRIMARY KEY unique.
- Données atomiques : firstname et lastname séparés, email distinct du mot de passe (hashed_password).
- Aucune dépendance transitive : product ne stocke pas le nom de la catégorie, il référence category_id. Le nom de la catégorie reste dans la table category.
- Les tables de liaison user_type et command_product évitent toute redondance pour les relations N-N.

Cle de conception : ON DELETE CASCADE partout

Chaque clé étrangère utilise ON DELETE CASCADE. Supprimer un utilisateur efface ses annonces, ses commandes et ses photos. Cela garantit l'intégrité référentielle sans intervention manuelle.

2. Implémentation Technique

2.1 Schéma SQL et Contraintes

Le script utilise des types de données optimisés et des contraintes d'intégrité fortes pour garantir la cohérence de la base de données.

Contrainte	Rôle et justification
ON DELETE CASCADE	Applique sur toutes les clés étrangères : suppression en cascade sur user, sale_ad, product, command, picture.
UNIQUE (email)	Garantit l'unicité des comptes : impossible de créer deux utilisateurs avec le même email.
NOT NULL	Oblige sur tous les champs critiques : title, price, quantity, hashed_password, date.
DECIMAL (10 , 2)	Type précis pour les prix (ex : 850.00). Evite les erreurs d'arrondi des float.
TIMESTAMP + DEFAULT	Horodatage automatique des commandes avec DEFAULT CURRENT_TIMESTAMP.

2.2 Jeu de données

Le jeu de données couvre tous les cas de test nécessaires :

- 10 utilisateurs : Jean Dupont (Admin + Vendeur + Acheteur), Marie Curie (top vendeuse), Emma Bernard (vendeuse sans vente), Lucas Martin (acheteur compulsif) et 6 autres profils variés.
- 10 catégories (Electronique, Maison, Loisirs, Vêtements...) et 10 produits dont 3 orphelins : Raquette de Tennis, Livre Harry Potter, Lego Batman — sans aucune annonce associée.
- 10 annonces dont 2 d'Emma Bernard (canape cuir et canape tissu) qui n'ont jamais été vendues — cas de test pour l'analyse métier.
- 10 commandes entre le 20 et 28 mars 2024, 15 photos réparties sur les annonces pour le carrousel UI.

Extension PostgreSQL
CREATE EXTENSION IF NOT EXISTS pg_trgm active la recherche textuelle de type 'Que recherchez-vous ?' de LeMauvaisCoin. Elle permet les recherches approximatives sur les titres d'annonces via l'index GIN trigram.

3. Requetes Metier Imposees

3.1 Jointures et Identification

Deux types de jointures répondent aux besoins fonctionnels de la marketplace :

INNER JOIN — Détail complet d'une annonce

Récupères-en une requête le titre, le produit, la catégorie, le prix et le nom du vendeur. C'est la requête utilisée pour afficher la page de détail d'une annonce sur le site.

```
SELECT sa.title AS annonce, p.name AS produit,
       c.name AS categorie, sa.price,
       u.firstname || ' ' || u.lastname AS vendeur
FROM sale_ad sa
INNER JOIN product p  ON sa.product_id = p.id
INNER JOIN category c ON p.category_id = c.id
INNER JOIN "user" u   ON sa.user_id = u.id;
```

LEFT JOIN — Produits orphelins (jamais mis en vente)

Identifie les produits référencés dans la base sans aucune annonce associée. Résultat : Raquette de Tennis, Livre Harry Potter, Lego Batman. Utile pour le marketing afin de cibler les catégories sous-représentées.

```
SELECT p.name AS produit_orphelin
FROM product p
LEFT JOIN sale_ad sa ON p.id = sa.product_id
WHERE sa.id IS NULL;
```

3.2 Agregats et Reporting

Les requêtes d'agrégation extraient des indicateurs métiers directement exploitables :

GROUP BY — Nombre d'annonces par categorie

Donne une vision du catalogue actif : combien d'annonces sont publiées dans chaque catégorie. Utile pour la page d'accueil et les filtres de navigation.

```
SELECT c.name AS categorie, COUNT(sa.id) AS nombre_annonces
FROM category c
JOIN product p ON c.id = p.category_id
```

```
JOIN sale_ad sa ON p.id = sa.product_id
GROUP BY c.name;
```

GROUP BY + HAVING — Power Users (multi-roles)

Identifie les utilisateurs ayant plus d'un rôle. Exemple : Jean Dupont est Admin + Vendeur + Acheteur (3 roles). Le HAVING filtre uniquement ceux qui ont COUNT > 1.

```
SELECT u.firstname, u.lastname, COUNT(ut.type_id) AS nb_roles
FROM "user" u
JOIN user_type ut ON u.id = ut.user_id
GROUP BY u.id, u.firstname, u.lastname
HAVING COUNT(ut.type_id) > 1;
```

Chiffre d'Affaires (CA) par utilisateur

Calcule le CA de chaque vendeur en joignant les commandes validées. Trie par ordre décroissant pour identifier les meilleurs vendeurs. Marie Curie ressort en tête.

```
SELECT u.firstname, u.lastname, SUM(sa.price) AS chiffre_affaires
FROM "user" u
JOIN sale_ad sa ON u.id = sa.user_id
JOIN command_product cp ON sa.id = cp.sale_ad_id
GROUP BY u.id, u.firstname, u.lastname
ORDER BY chiffre_affaires DESC;
```

3.3 Transaction Metier

Une transaction complète simule un achat sécurise en 3 étapes atomiques :

1. Création de la commande (INSERT INTO command avec CURRENT_TIMESTAMP).
2. Liaison à l'annonce achetée (INSERT INTO command_product avec LASTVAL()).
3. Décrémentation du stock (UPDATE sale_ad SET quantity = quantity - 1).

Garantie d'intégrité : BEGIN / COMMIT

Si l'une des 3 étapes échoue, toute l'opération est annulée (ROLLBACK implicite). Le stock n'est décrémenté que si la commande est bien enregistrée. LASTVAL() récupère l'ID de la dernière commande insérée sans avoir besoin de le passer manuellement.

```
BEGIN;

INSERT INTO command (date, user_id)
VALUES (CURRENT_TIMESTAMP, 3);
```

```
INSERT INTO command_product (command_id, sale_ad_id)
VALUES (LASTVAL(), 3);

UPDATE sale_ad SET quantity = quantity - 1
WHERE id = 3;

COMMIT;
```

4. Performance et Optimisation

L'analyse des performances est réalisée via EXPLAIN, qui révèle le plan d'exécution du moteur PostgreSQL. Nous avons défini 9 index couvrant 4 cas d'usage distincts du site.

4.1 Index CA — Chiffre d'Affaires par vendeur

Avant index	Index applique	Apres index
Sequential Scan sur command_product — lecture ligne par ligne de toute la table de liaison.	<pre>CREATE INDEX idx_cp_sale_ad ON command_product(sale_ad_id);</pre>	Index Scan sur idx_cp_sale_ad — acces direct aux lignes matchantes, jointures optimisees.

4.2 Index filtre prix + produit (barre de filtre LeMauvaisCoin)

Le filtrage croise prix / catégorie est la requête la plus fréquente sur une marketplace. Un index composite couvre les deux critères simultanément :

```
-- AVANT : Full Table Scan sur sale_ad
EXPLAIN SELECT * FROM sale_ad WHERE price > 500 AND product_id = 1;

-- INDEX COMPOSITE (product_id, price) — ordre important
CREATE INDEX idx_sa_product_price ON sale_ad(product_id, price);

-- APRES : Index Scan cible — filtre sur les 2 colonnes en une passe
EXPLAIN SELECT * FROM sale_ad WHERE price > 500 AND product_id = 1;
```

Pourquoi (product_id, price) et pas l'inverse ?

PostgreSQL peut utiliser les premières colonnes d'un index composite pour filtrer seul. Mettre product_id en premier permet de filtrer d'abord sur la catégorie (cardinalité élevée), puis restreindre par prix. L'inverse serait moins efficace car price est continu.

4.3 Index recherche textuelle (barre de recherche)

La barre 'Que recherchez-vous ?' de LeMauvaisCoin nécessite une recherche approximative sur les titres. L'extension `pg_trgm` permet un index GIN trigram :

```
-- Index trigram pour la recherche de type LIKE '%iphone%'
CREATE INDEX idx_sale_ad_title_trgm
  ON sale_ad USING GIN (title gin_trgm_ops);

-- Exemple d'utilisation (recherche insensible a la casse)
SELECT * FROM sale_ad
  WHERE title ILIKE '%iphone%';
```

4.4 Index de performance générale

6 index supplémentaires couvrent les clés étrangères et le tri par défaut (annonces les plus récentes en premier) :

```
-- Tri par défaut : annonces les plus recentes d'abord
CREATE INDEX idx_sale_ad_id_desc ON sale_ad(id DESC);

-- Filtrage croise annonce <-> categorie
CREATE INDEX idx_product_category_id ON product(category_id);

-- Performances generales (cles etrangeres)
CREATE INDEX idx_sale_ad_user_id    ON sale_ad(user_id);
CREATE INDEX idx_sale_ad_product_id ON sale_ad(product_id);
CREATE INDEX idx_command_user_id    ON command(user_id);
CREATE INDEX idx_picture_sale_ad_id ON picture(sale_ad_id);
```

Bilan : 9 index au total

`idx_sale_ad_title_trgm` (GIN), `idx_sale_ad_price`, `idx_product_category_id`, `idx_sale_ad_id_desc`, `idx_cp_sale_ad_id`, `idx_sa_product_price` (composite), `idx_sale_ad_user_id`, `idx_sale_ad_product_id`, `idx_command_user_id`, `idx_picture_sale_ad_id`. Chaque index répond a un cas d'usage concret du site.

5. Analyse Métier (Interprétation)

L'exploitation des requêtes permet de dégager des insights concrets sur l'activité de la marketplace.

5.1 Analyse des produits inactifs

Constat : 3 produits sur 10 n'ont aucune annonce (30%)

La requête LEFT JOIN révèle que Raquette de Tennis (Loisirs), Livre Harry Potter (Livres & BD) et Lego Batman (Jouets) sont référencés dans le catalogue mais n'ont jamais été mis en vente. Ces catégories sont donc totalement inactives.

Action suggérée

Envoyer des notifications aux vendeurs potentiels pour encourager les dépôts dans ces catégories. Le catalogue actif passerait de 7 à 10 catégories.

5.2 Rentabilité des vendeurs

Constat : Emma Bernard a 2 annonces sans aucune vente

Canape cuir (300 euros, annonce 3) et Canape tissu (100 euros, annonce 9) d'Emma Bernard ne figurent dans aucune `command_product`. Sa requête CA retourne NULL ou 0, contrairement à Marie Curie qui cumule les ventes iPhone (850 euros) et MacBook (1100 euros).

Action suggérée

Analyser si le problème vient du prix (300 euros pour un canape avec tache), de la description ou de la visibilité. Proposer à Emma Bernard un outil de suggestion de prix basé sur les ventes comparables.

Conclusion

Ce projet illustre la conception complète d'un système de gestion de données relationnel de type marketplace, de la modélisation initiale jusqu'à l'optimisation fine des performances. Les quatre axes démontres lors de la soutenance sont :

OK	Schema solide (3NF) — 9 tables, clés étrangères CASCADE, types DECIMAL et TIMESTAMP, extension <code>pg_trgm</code> .
OK	Jeu de données réaliste — 10 utilisateurs, 10 produits (3 orphelins), 10 annonces (2 sans vente), 15 photos, 10 commandes.
OK	Transactions fiables — BEGIN / LASTVAL() / COMMIT garantit l'intégrité des stocks à chaque achat.
OK	9 index stratégiques — GIN trigram, composite (<code>product_id</code> , <code>price</code>), DESC pour la pagination — chacun lié à un cas d'usage réel du site.